



**Faculty of Engineering and Technology
Electrical and Computer Engineering Department**

**ENCS3390: Operating System Concepts
ENCS4110**

Report Project #1

Prepared by:

Student Name: Aya Abdullah

Student NO: 1220782

Instructor: Mohammad Khalil

Section: 2

Date: 8-12-2024.

Abstract

Project investigates three different approaches—naive, multiprocessing, and multithreading—for efficiently computing word frequencies from a large dataset. The naive approach executes tasks sequentially, making it inefficient for large datasets. The multiprocessing approach divides the workload across multiple processes, utilizing several CPU cores for parallel execution, significantly reducing execution time. Similarly, the multithreading approach achieves concurrency by dividing the work into threads within a single process, which provides improvements over the naive approach but is less effective than multiprocessing in terms of performance. This paper evaluates the performance of each approach based on execution time, speedup, and throughput, taking into account the system's hardware and the inherent limitations of parallelization. The optimal configurations for multiprocessing and multithreading are identified, and the impact of system resources on performance is discussed, with multiprocessing showing the most significant performance gains.

Table of Contents

Abstract.....	1
Table of Contents.....	2
Theory	3
Environment Description	4
Discussion and Analysis.....	4
Performance Evaluation.....	11
Performance Analysis Based on Amdahl's Law.....	11
Determining the Optimal Number of Processes or Threads.....	13
Achieving Maximum Speedup.....	14
Out figures.....	17
Coclusion.....	20

1. Theory

1.Naïve Approach

The naive approach involves completing tasks one at a time, without using parallel processing. This method is simple and easy to implement, making it suitable for small tasks. However, it cannot speed up execution since tasks are handled one after another. As a result, it is not ideal for complex or time-consuming tasks that could benefit from running in parallel.

2.Multiprocessing Approach

The multiprocessing approach uses multiple processes to run tasks at the same time. Each process works independently and has its own memory. This method is useful for heavy tasks that can be divided into smaller, separate parts. It can significantly speed up performance by using multiple CPU cores but has higher overhead due to the need for processes to communicate and manage memory separately.

3.Multithreading Approach

The multithreading approach runs multiple threads within a single process, where all threads share the same memory. Tasks are divided into smaller parts, and each part is run by a different thread. This method is faster and uses fewer resources than multiprocessing. However, it requires careful management to avoid errors like data conflicts or crashes. It is best for tasks that need frequent access to shared data and can benefit from splitting into smaller, connected parts.

2. Environment Description

Computer Specifications

- **Cores:** The computer has **6 cores** with **6 threads**
- **Speed:** The CPU model is the **Intel Core i7-1255U** (12th Gen), which typically has a base clock speed of around **2.7 GHz**.
- **Memory:** system has **4.8 GB of RAM** in total, with **1.0 GB currently in use, and 2.2 GB free**. The swap memory, which acts as virtual memory when the physical RAM is full, is unused at the moment, with all 2.6 GB available.

Operating System

- The system is running **Ubuntu 22.04.2 LTS**, a popular Linux distribution, in a **virtual machine** (as indicated by Oracle VirtualBox).

Programming Language and IDE Tool

- **Programming Language:** The project is developed **using C language**.
- **IDE Tool:** The IDE used is **Code::Blocks**, which is open-source, cross-platform IDE specifically designed for C, C++.

Virtual Machine

- **Virtual Machine Usage:** the environment is running in a **virtual machine** managed by **Oracle VirtualBox**.

Discussion and Analysis

➤ 1.Part One: Defining STRUCT to store input data

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define INITIAL_CAPACITY 1024 // Initial capacity for dynamic a
#define MAX_WORD_LENGTH 50   // Maximum length of each word

typedef struct {
    char word[MAX_WORD_LENGTH];
    int frequency;
} WordFreq;
```

This part of the code defines a structure (**WordFreq**) that stores two attributes: a string (**word**) to hold each word and an integer (**frequency**) to store the word's frequency in the dataset. The **MAX_WORD_LENGTH** constant limits the length of each word to **50 characters**, ensuring memory usage is controlled and optimized. This structure will be used later in the program to store and manipulate word frequencies during the text processing phase.

➤ Part two : read from file and store data to array :

```
int read_words(const char *filename, char ***words, int *word_count) {
    clock_t start_time = clock(); // Start timing

    FILE *file = fopen(filename, "r");
    if (!file) {
        fprintf(stderr, "Error: Could not open file '%s'.\n", filename);
        return -1;
    }

    int capacity = INITIAL_CAPACITY;
    *words = malloc(capacity * sizeof(char *));
    if (!*words) {
        fprintf(stderr, "Memory allocation failed for words array.\n");
        fclose(file);
        return -1;
    }

    char buffer[MAX_WORD_LENGTH];
    *word_count = 0;
    while (fscanf(file, "%49s", buffer) == 1) {
        grow_array_if_needed(word_count, &capacity, (void **)words, sizeof(char *));
        (*words)[*word_count] = malloc(strlen(buffer) + 1);
        if (!(*words)[*word_count]) {
            fprintf(stderr, "Memory allocation failed for word '%s'.\n", buffer);
            fclose(file);
            return -1;
        }
        strcpy((*words)[*word_count], buffer);
        (*word_count)++;
    }

    fclose(file);

    clock_t end_time = clock(); // End timing
    double time_taken = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
    printf("Time taken in read_words: %.6f seconds\n", time_taken); // Print the time

    return 0;
}

void grow_array_if_needed(int *count, int *capacity, void **array, size_t element_size) {
    clock_t start_time = clock(); // Start timing

    if (*count >= *capacity) {
        *capacity *= 2; // Double the capacity
        void *new_array = realloc(*array, (*capacity) * element_size);
        if (!new_array) {
            fprintf(stderr, "Memory allocation failed during resizing.\n");
            exit(EXIT_FAILURE);
        }
        *array = new_array;
    }

    clock_t end_time = clock(); // End timing
    double time_taken = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
}
```

Analysis:

This code snippet demonstrates the process of reading words from the "data.txt" file and storing them in a dynamically allocated array. The **fscanf** function is used to read individual words, with a limit on the number of characters to prevent overflow (defined by **MAX_WORD_LENGTH**).

The **grow_array_if_needed** function ensures that the array dynamically expands as needed. If the array reaches its capacity, it doubles in size, which reduces the frequency of reallocations and improves memory efficiency.

Each word is stored in a dynamically allocated array of strings, with the **malloc** function used for memory allocation. The total word count is returned through the **word_count** variable for further processing.

- *Naive Approach:*

Code:

```
// Calculate word frequencies
int calculate_frequencies(char **words, int word_count, WordFreq **frequencies, int *unique_count) {
    int capacity = INITIAL_CAPACITY;
    *frequencies = malloc(capacity * sizeof(WordFreq));
    if (!*frequencies) {
        fprintf(stderr, "Memory allocation failed for word frequencies.\n");
        return -1;
    }

    *unique_count = 0;
    for (int i = 0; i < word_count; i++) {
        int found = 0;
        for (int j = 0; j < *unique_count; j++) {
            if (strcmp(words[i], (*frequencies)[j].word) == 0) {
                (*frequencies)[j].frequency++;
                found = 1;
                break;
            }
        }
        if (!found) {
            grow_array_if_needed(unique_count, &capacity, (void **)frequencies, sizeof(WordFreq));
            strcpy((*frequencies)[*unique_count].word, words[i]);
            (*frequencies)[*unique_count].frequency = 1;
            (*unique_count)++;
        }
    }

    return 0;
}

// main
char **words;
WordFreq *frequencies;
int word_count, unique_count;

// Read words from the file
if (read_words("data.txt", &words, &word_count) < 0) {
    return EXIT_FAILURE;
}
printf("Total number of words: %d\n", word_count);

// Start timing
clock_t start_time = clock();

// Calculate word frequencies
if (calculate_frequencies(words, word_count, &frequencies, &unique_count) < 0) {
    free_words(words, word_count);
    return EXIT_FAILURE;
}

// End timing
clock_t end_time = clock();
double time_taken = ((double)(end_time - start_time)) / CLOCKS_PER_SEC;
printf("Number of unique words: %d\n", unique_count);
printf("Time taken to calculate frequencies: %.6f seconds\n", time_taken);

// Sort and print top 10 most frequent words
qsort(frequencies, unique_count, sizeof(WordFreq), compare_frequencies);
print_top_frequencies(frequencies, unique_count, 10);

// Free allocated memory
free_words(words, word_count);
free_frequencies(frequencies);
```

The provided code implements a naive approach to calculating word frequencies from a file. It reads words and stores them in a dynamically allocated array. The program iterates over each word and checks if it already exists in the frequency array. If found, its frequency is incremented; otherwise, it is added to the array. The approach involves a linear search for each word, making the time complexity of frequency calculation $O(n * m)$, where n is the total number of words and m is the number of unique words. Sorting the frequencies has a complexity of $O(m * \log(m))$. The overall time complexity is $O(n * m)$, which can become $O(n^2)$ in the worst case when all words are unique. The space complexity is $O(n)$ for storing the words and $O(m)$ for storing unique word frequencies. This naive approach is inefficient for large datasets with many unique words.

➤ Multiprocessing Approach :

Code :

```
WordFreq *frequencies = NULL;
int word_count, unique_count = 0;
int num_processes = 2;

if (read_words("data.txt", &words, &word_count) < 0) {
    return EXIT_FAILURE;
}
printf("Total number of words: %d\n", word_count);
clock_t process_start_time = clock();

pid_t pids[num_processes];
int words_per_process = word_count / num_processes;

for (int i = 0; i < num_processes; i++) {
    pids[i] = fork();
    if (pids[i] == 0) {
        int start = i * words_per_process;
        int end = (i == num_processes - 1) ? word_count : start + words_per_process;
        calculate_frequencies_in_child(words, start, end, i);
        exit(0);
    }
}

for (int i = 0; i < num_processes; i++) {
    wait(NULL);
}

clock_t process_end_time = clock();
printf("Time taken by parent process: %.6f seconds\n", (double)(process_end_time - process_start_time) / CLOCKS_PER_SEC);

if (merge_frequencies(&frequencies, &unique_count, num_processes) < 0) {
    free_words(words, word_count);
    return EXIT_FAILURE;
}

qsort(frequencies, unique_count, sizeof(WordFreq), compare_frequencies);
print_top_frequencies(frequencies, unique_count, 10);

free_words(words, word_count);
free_frequencies(frequencies);

return EXIT_SUCCESS;

void calculate_frequencies_in_child(char **words, int start, int end, int process_id) {
    clock_t start_time = clock();

    WordFreq *frequencies = malloc(INITIAL_CAPACITY * sizeof(WordFreq));
    int unique_count = 0;
    int capacity = INITIAL_CAPACITY;

    for (int i = start; i < end; i++) {
        int found = 0;
        for (int j = 0; j < unique_count; j++) {
            if (strcmp(words[i], frequencies[j].word) == 0) {
                frequencies[j].frequency++;
                found = 1;
                break;
            }
        }
        if (!found) {
            grow_array_if_needed(&unique_count, &capacity, (void **)&frequencies, sizeof(WordFreq));
            strcpy(frequencies[unique_count].word, words[i]);
            frequencies[unique_count].frequency = 1;
            unique_count++;
        }
    }

    char filename[20];
    snprintf(filename, sizeof(filename), "frequencies_%d.txt", process_id);
    FILE *file = fopen(filename, "w");
    if (!file) {
        fprintf(stderr, "Error: Could not open file '%s' for writing.\n", filename);
        exit(EXIT_FAILURE);
    }

    for (int i = 0; i < unique_count; i++) {
        fprintf(file, "%s %d\n", frequencies[i].word, frequencies[i].frequency);
    }

    fclose(file);
    free(frequencies);

    clock_t end_time = clock();
    printf("Time taken by process %d: %.6f seconds\n", process_id, (double)(end_time - start_time) / CLOCKS_PER_SEC);
}
```

```

int merge_frequencies(MordFreq **frequencies, int *unique_count, int num_processes) {
    clock_t start_time = clock();

    int capacity = INITIAL_CAPACITY;
    *frequencies = malloc(capacity * sizeof(MordFreq));
    if (!*frequencies) {
        fprintf(stderr, "Memory allocation failed for merged frequencies.\n");
        return -1;
    }

    for (int i = 0; i < num_processes; i++) {
        char filename[20];
        snprintf(filename, sizeof(filename), "frequencies_%d.txt", i);

        FILE *file = fopen(filename, "r");
        if (!file) {
            fprintf(stderr, "Error: Could not open file '%s' for reading.\n", filename);
            return -1;
        }

        char word[MAX_WORD_LENGTH];
        int frequency;
        while (fscanf(file, "%49s %d", word, &frequency) == 2) {
            int found = 0;
            for (int j = 0; j < *unique_count; j++) {
                if (strcmp(word, (*frequencies)[j].word) == 0) {
                    (*frequencies)[j].frequency += frequency;
                    found = 1;
                    break;
                }
            }
            if (!found) {
                grow_array_if_needed(unique_count, &capacity, (void **)frequencies, sizeof(MordFreq));
                strcpy((*frequencies)[*unique_count].word, word);
                (*frequencies)[*unique_count].frequency = frequency;
                (*unique_count)++;
            }
        }
        fclose(file);
    }

    clock_t end_time = clock();
    printf("Time taken in merge_frequencies: %.6f seconds\n", (double)(end_time - start_time) / CLOCKS_PER_SEC);
    return 0;
}

```

The multiprocessing requirements in the code are achieved using various techniques and standard APIs to ensure efficient parallel processing of word frequency analysis. The **fork()** system call is central to this implementation, creating child processes that independently handle portions of the dataset. Each child process processes a specific range of words, calculated based on evenly dividing the dataset, and executes the **calculate_frequencies_in_child** function. This function computes word frequencies for the assigned segment and stores the results in uniquely named files using **snprintf()** to prevent conflicts (frequencies_0.txt, frequencies_1.txt). The parent process ensures proper synchronization by invoking **wait()** after initiating all child processes, blocking further execution until all children complete their tasks. Once the child processes finish, the parent process calls **the merge_frequencies function to combine results from the individual output files into a single**, consolidated frequency array. The merging process uses dynamic memory management with **realloc()** to handle growing data and avoids conflicts by iteratively updating the global frequency list. After merging, the results are sorted using **qsort()**, enabling efficient ranking of words based on frequency. By employing APIs like **fork()**, **wait()**, **snprintf()**, **realloc()**, and **qsort()**, the program leverages multiple CPU cores for parallel execution, achieving significant performance improvements when processing large datasets. This approach efficiently distributes the workload, ensures proper synchronization, and consolidates the results for analysis, demonstrating a robust and scalable solution for word frequency computation.

➤ *MultiThreading Approach :*

Code

```
typedef struct {
    char **words;
    int start_index;
    int end_index;
    WordFreq **local_frequencies;
    int *local_unique_count;
    int *local_capacity;
    double start_time;
    double end_time;
} ThreadData;

sem_t frequency_semaphore; // Semaphore for synchronizing access to global frequencies

// Function to calculate frequencies for a thread
void *calculate_frequencies_thread(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    data->start_time = get_time_seconds(); // Start time for thread

    double func_start_time = get_time_seconds(); // Start time for calculate_frequencies_thread
    int local_unique_count = 0;
    WordFreq *local_frequencies = malloc(INITIAL_CAPACITY * sizeof(WordFreq));

    for (int i = data->start_index; i < data->end_index; i++) {
        int found = 0;
        for (int j = 0; j < local_unique_count; j++) {
            if (strcmp(local_frequencies[j].word, data->words[i]) == 0) {
                local_frequencies[j].frequency++;
                found = 1;
                break;
            }
        }
        if (!found) {
            grow_array_if_needed(&local_unique_count, data->local_capacity, (void **)&local_frequencies, sizeof(WordFreq));
            strcpy(local_frequencies[local_unique_count].word, data->words[i]);
            local_frequencies[local_unique_count].frequency = 1;
            local_unique_count++;
        }
    }

    data->end_time = get_time_seconds(); // End time for thread
    data->local_frequencies = local_frequencies;
    data->local_unique_count = local_unique_count;

    double func_end_time = get_time_seconds(); // End time for calculate_frequencies_thread
    printf("Function 'calculate_frequencies_thread' took %.4f seconds.\n", func_end_time - func_start_time);

    return NULL;
}
```

```

nt word_count, unique_count;

f (read_words("data.txt", &words, &word_count) < 0) {
    return EXIT_FAILURE;
}

printf("Total number of words: %d\n", word_count);

frequencies = malloc(INITIAL_CAPACITY * sizeof(WordFreq));
unique_count = 0;
nt capacity = INITIAL_CAPACITY;

sem_init(&frequency_semaphore, 0, 1); // Initialize semaphore

thread_t threads[NUM_THREADS];
nt chunk_size = word_count / NUM_THREADS;

hreadData thread_data[NUM_THREADS];
or (int i = 0; i < NUM_THREADS; i++) {
    thread_data[i].words = words;
    thread_data[i].start_index = i * chunk_size;
    thread_data[i].end_index = (i == NUM_THREADS - 1) ? word_count : (i + 1) * chunk_size;
    thread_data[i].local_frequencies = NULL;
    thread_data[i].local_unique_count = 0;
    thread_data[i].local_capacity = malloc(sizeof(int));
    *thread_data[i].local_capacity = INITIAL_CAPACITY;
    pthread_create(&threads[i], NULL, calculate_frequencies_thread, &thread_data[i]);

or (int i = 0; i < NUM_THREADS; i++) {
    pthread_join(threads[i], NULL);
    merge_frequencies(&frequencies, &unique_count, &capacity, thread_data[i].local_frequencies, thread_data[i].local_unique_count);

sort(frequencies, unique_count, sizeof(WordFreq), compare_frequencies);
rint_top_frequencies(frequencies, unique_count, 10);

etimeofday(&end_time, NULL); // End time for the whole program
ouble total_time = (end_time.tv_sec - start_time.tv_sec) + (end_time.tv_usec - start_time.tv_usec) / 1000000.0;
rintf("Total program execution time: %.4f seconds.\n", total_time);

```

In this code, **Multithreading** is achieved using the **Pthreads** library, where multiple threads are created to process different chunks of the dataset concurrently. The **pthread_create** function is used to initiate threads, and each thread is assigned a specific portion of the word list. Once all threads have been created, **the pthread_join** function is used to ensure the main thread waits for all threads to finish before proceeding with the next step. Each thread computes word frequencies locally for its assigned data segment, and these local results are then merged into a global frequency list. To manage synchronization and avoid race conditions, **a semaphore** is used. The semaphore, initialized **with sem_init**, ensures that only one thread can modify the global frequency array at a time by using **sem_wait** to acquire the lock and **sem_post** to release it once the update is complete. This allows the program to merge the results safely from multiple threads. Additionally, **memory management** is handled dynamically using **realloc** to resize arrays as more data is processed. The execution time of each section of the program, such as reading the data, processing frequencies, and merging results, is measured using the **gettimeofday** function. This approach ensures that the workload is efficiently distributed, significantly speeding up the process while maintaining thread safety during the merging phase.

Performance Evaluation and Analysis:

➤ Performance evaluation

in this code measures the execution time and throughput of the multiprocessing approach with varying numbers of processes. As the number of processes increases, execution time should decrease due to parallel processing, improving efficiency. However, performance gains may diminish after a certain point due to process creation and communication overhead.

Throughput is defined as **Performance (Throughput) = 1 / Time**.

➤ Performance Analysis According to Amdahl's Law:

Amdahl's Law provides a way to estimate the maximum theoretical speedup of a program when part of it is parallelized. The law is expressed as:

$$\text{Speedup} = \frac{1}{s + (1-s)/N}$$

- S: is the serial part of the program,
- N is the number of processors or threads.

In the program, the tasks that can be parallelized are primarily the frequency calculation (**calculate_frequencies_thread**) and the merging of local frequency results (**merge_frequencies**). These portions of the code were divided across multiple threads to take advantage of parallel processing.

However, certain tasks must be executed serially, including:

- **Reading words from the file** (read_words function), which is inherently serial as the data is processed sequentially.
- **Sorting the frequency results** (qsort), which remains serial in this implementation.
- **Printing the top frequencies to the console** (print_top_frequencies), which also needs to be done serially.
- **Free function**

Thus, the serial portion of the program is mainly due to file reading, sorting, and printing operations. we estimate that approximately **.4%** of the code is serial, and **99.6%** of the code can be parallelized.

Calculate serial part :

Execution time : 917.1 s

```
Process returned 0 (0x0)   execution time : 917.100 s
Press ENTER to continue.
```

This is the parallel time : 913.564 s

```
Total number of words: 17005214
Number of unique words: 253858
Time taken to calculate frequencies: 913.563933 seconds
Top 10 most frequent words:
```

Parallel portion : $\text{parallel time} / \text{execution time} * 100\% = 99.6\%$

So the serial portion = 0.4%

➤ Table of Comparison:

Type of approach	Time execution(S)	SPED UP	PERFORMANCE
Naïve approach	917.1	1	.00109
(Multiprocess)2 processes	572.519	1.6	.00175
(Multi process)4 processes	352.085	2.6	.00284
(Multiprocess)6 processes	313.683	2.92	.00319
(Multiprocess)8 processes	328.642	2.79	.00304
(Multithread) 2 threads	384.050	2.39	.00260
(Multithread) 4 threads	321.578	2.85	.00311
(Multithread) 6 threads	348.461	2.63	.00287
(Multithread) 8 threads	438.628	2.09	.00228

➤ Optimal Number of Processes or Threads:

From the performance data in the table:

- **Processes:** The best performance is achieved **with 6 processes**, achieving an execution time of 313.683 s and a speedup of 2.92. Increasing to **8 processes** results in slightly higher execution time (328.642 s) due to overhead. This behavior is expected since the PC has only 6 physical cores, and adding more processes introduces overhead due to context switching and contention for CPU resources.
- **Threads:**
 - The optimal performance is observed with **4 threads**, achieving an execution time of 321.578 s and a speedup of 2.85.
 - Using **6 threads** (matching the number of CPU threads) yields a speedup of 2.63. However, the performance starts degrading at **8 threads** (execution time 438.628 s) because the system exceeds the available CPU threads, leading to significant resource contention and scheduling overhead.

These results highlight that the system's hardware constraints (6 cores and 6 threads) significantly affect the outcomes when the parallelization exceeds the hardware capacity.

➤ maximum speedup:

From your previous analysis, you have determined that the serial portion of the code (S) is **0.4%** or **0.004**.

The maximum speedup depends on the number of cores available on your system. As per your setup, your system has **6 cores**.

Now, using Amdahl's Law, we can calculate the maximum speedup for **6 cores**:

speedup max = $\frac{1}{.004 + (1 - .004)/6}$ Thus, the **maximum theoretical speedup** according to **6 cores** is approximately **5.88**. This is the best achievable speedup if there were no other factors (such as overhead or inefficiencies) limiting the parallel execution.

➤ Performance Comparison :

Naive Approach:

The **naive approach** serves as the baseline for comparison, where the entire program runs sequentially without any parallelization. It has the longest execution time of **917.1 seconds** and a performance of **0.00109**. This approach is fundamentally limited as every task, including file reading, frequency calculation, sorting, and printing, is processed one after the other. Since all tasks are handled serially, there is no concurrency or parallel execution to take advantage of modern multi-core processors. The lack of parallelism significantly increases the total execution time, especially when processing large datasets. This approach highlights the inefficiencies of running everything sequentially, making it an ideal point of reference for comparing the other two parallelized approaches.

Multiprocessing Approach:

The **multiprocessing approach** significantly reduces execution time by splitting the task into multiple processes, allowing the workload to be distributed across multiple CPU cores. As the number of processes increases, there is a marked reduction in execution time. For example, with **two processes**, the execution time reduces to **572.519 seconds**, improving performance to **0.00175**. This trend continues as we increase the number of processes. With **four processes**, the execution time further decreases to **352.085 seconds**, with a performance of **0.00284**, and with **six processes**, the time reduces to **313.683 seconds**, resulting in a performance of **0.00319**.

However, as the number of processes exceeds six, the performance begins to level off. When **eight processes** are used, the execution time slightly increases to **328.642 seconds**, with a performance of **0.00304**. This is indicative of the law of diminishing returns in parallel computing. While the parallelization helps distribute the work, the overhead associated with managing more processes (e.g., process synchronization, inter-process communication) leads to a slight increase in execution time. In essence, the system is likely reaching the limit of its capacity to efficiently handle the added processes, thus reducing the overall speedup.

The multiprocessing approach benefits significantly from parallelism, but after a certain point, adding more processes introduces overhead that negates the gains from additional parallelism. This trend suggests that the ideal number of processes is likely **six** for optimal performance in this particular scenario.

Multithreading Approach:

The **multithreading approach** also provides parallelism but with a different mechanism. Instead of creating separate processes, threads are created within a single process, allowing for shared memory access and lighter resource usage compared to processes. The performance

improvements with multithreading are similar to multiprocessing but with less drastic reductions in execution time.

With **two threads**, the execution time is **384.050 seconds**, resulting in a performance of **0.00260**. As more threads are introduced, the performance improves: with **four threads**, the execution time drops to **321.578 seconds** and a performance of **0.00311**. However, the improvement starts to plateau as we add more threads. With **six threads**, the execution time increases slightly to **348.461 seconds**, and with **eight threads**, it further increases to **438.628 seconds**. The performance for these configurations is lower than with the multiprocessing approach, suggesting that threads, while useful, do not provide as significant a speedup as processes, especially in cases where the workload can benefit more from separate processes with their own memory space.

The higher-than-expected execution times for **six** and **eight threads** suggest that the system might be reaching its limits, and having more threads than available CPU cores (which is **6 cores** and **12 threads**) results in contention for resources, leading to increased overhead. The multithreading approach may be more efficient in scenarios with fewer threads or tasks that require less synchronization and can share memory effectively, but when threads exceed the number of cores, they may cause performance degradation due to excessive context switching and resource contention.

The performance analysis reveals that **multiprocessing** provides the best performance improvement over the **naive approach**, with a significant reduction in execution time as the number of processes increases. The optimal number of processes is **6**, as adding more processes beyond this results in diminishing returns due to increased management overhead. **Multithreading** also improves performance compared to the naive approach, but not as effectively as multiprocessing. The optimal number of threads is **4**, after which performance degrades due to thread contention. Overall, **multiprocessing** is the most efficient approach for this task, while **multithreading** offers limited benefits. The **naive approach** is the least efficient, especially for large datasets.

Output figures:

➤ Naïve part:

```
OS
Total number of words: 17005214
Number of unique words: 25398
Time taken to calculate frequencies: 933.563033 seconds
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644

Process returned 0 (0x0)   execution time : 937.100 s
Press ENTER to continue.
```

➤ Multiprocessing :

➤ 2 processes:

```
Time taken in read_words: 3.590630 seconds
Total number of words: 17005214
Time taken by process 0: 374.261154 seconds
Time taken by process 1: 374.343440 seconds
Time taken by parent process: 0.049505 seconds
Time taken in merge_frequencies: 193.314164 seconds
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Time taken in print_top_frequencies: 0.000057 seconds
Process returned 0 (0x0)   execution time : 572.519 s
Press ENTER to continue.
```

```
172
173 void free words(char **w
174     for (int i = 0; i <
175         free(words[i]);
```

➤ 4 processes :

```
OS
Time taken in read_words: 2.460005 seconds
Total number of words: 17005214
Time taken by process 2: 153.814986 seconds
Time taken by process 1: 156.998547 seconds
Time taken by process 0: 164.125393 seconds
Time taken by process 3: 170.379586 seconds
Time taken by parent process: 0.149542 seconds
Time taken in merge_frequencies: 177.806644 seconds
Top 10 most frequent words:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Time taken in print_top_frequencies: 0.000091 seconds
Process returned 0 (0x0)   execution time : 352.085 s
Press ENTER to continue.
```

```
197 pid_t pids[num_processes];
198 int words_per_process = word_count
199
```

➤ 6 processes :

```

Time taken by process 1: 108.972439 seconds
Time taken by process 0: 112.636721 seconds
Time taken by process 2: 112.109882 seconds
Time taken by process 4: 114.820618 seconds
Time taken by process 3: 114.531073 seconds
Time taken by process 5: 114.809031 seconds
Time taken by parent process: 0.112235 seconds
Time taken in merge_frequencies: 192.739174 seconds
Top 10 most frequent words:
the: 1061336
of: 593677
and: 416629
one: 411764
in: 372201
at: 326873
to: 316376
zero: 264975
nine: 250430
two: 132644
Time taken in print_top_frequencies: 0.000042 seconds
Process returned 0 (0x0)  execution time : 313.683 s
Press ENTER to continue.

```

➤ 8 processes

```

Time taken by process 1: 108.972439 seconds
Time taken by process 0: 112.636721 seconds
Time taken by process 2: 112.109882 seconds
Time taken by process 4: 114.820618 seconds
Time taken by process 3: 114.531073 seconds
Time taken by process 5: 114.809031 seconds
Time taken by parent process: 0.112235 seconds
Time taken in merge_frequencies: 192.739174 seconds
Top 10 most frequent words:
the: 1061336
of: 593677
and: 416629
one: 411764
in: 372201
at: 326873
to: 316376
zero: 264975
nine: 250430
two: 132644
Time taken in print_top_frequencies: 0.000042 seconds
Process returned 0 (0x0)  execution time : 313.683 s
Press ENTER to continue.

```

➤ Multithreading:

➤ 2 threads:

```

Function 'read_words' took 2.1030 seconds.
Total number of words: 17005214
Function 'calculate_frequencies.thread' took 228.4143 seconds.
Function 'calculate_frequencies.thread' took 231.7305 seconds.
Function 'merge_frequencies' took 48.5388 seconds.
Function 'merge_frequencies' took 103.6232 seconds.
Top 10 most frequent words:
the: 1061336
of: 593677
and: 416629
one: 411764
in: 372201
at: 326873
to: 316376
zero: 264975
nine: 250430
two: 132644
Function 'print_top_frequencies' took 0.0002 seconds.
Total program execution time: 383.6782 seconds.
Function 'free_words' took 0.3808 seconds.
Function 'free_frequencies' took 0.0009 seconds.
Process returned 0 (0x0)  execution time : 384.050 s
Press ENTER to continue.

```

➤ 4 threads:

```

Function 'read_words' took 1.8167 seconds.
Total number of words: 1709214
Function 'calculate_frequencies_thread' took 141.6448 seconds.
Function 'calculate_frequencies_thread' took 141.6203 seconds.
Function 'calculate_frequencies_thread' took 144.2943 seconds.
Function 'calculate_frequencies_thread' took 156.4345 seconds.
Function 'merge_frequencies' took 23.3576 seconds.
Function 'merge_frequencies' took 41.1383 seconds.
Function 'merge_frequencies' took 50.7162 seconds.
Function 'merge_frequencies' took 62.4492 seconds.
Top 10 most frequent words:
one: 938677
of: 489253
and: 411784
in: 372201
is: 326973
the: 316376
was: 264976
were: 250430
but: 153544
Function 'print_top_frequencies' took 0.0003 seconds.
Total program execution time: 321.1912 seconds.
Function 'free_words' took 0.0279 seconds.
Function 'free_frequencies' took 0.0004 seconds.
Process returned 0 (0x0)   execution time : 321.578 s
Press ENTER to continue.

```

➤ 8 threads

```

#define MAX_WORD_LENGTH 50
typedef struct {
    char word[MAX_WORD_LENGTH];
    int frequency;
} Word;

```

```

Function 'read_words' took 4.2895 seconds.
Total number of words: 1709214
Function 'calculate_frequencies_thread' took 137.0796 seconds.
Function 'calculate_frequencies_thread' took 141.2125 seconds.
Function 'calculate_frequencies_thread' took 142.5485 seconds.
Function 'calculate_frequencies_thread' took 143.5937 seconds.
Function 'calculate_frequencies_thread' took 146.4110 seconds.
Function 'calculate_frequencies_thread' took 146.4657 seconds.
Function 'calculate_frequencies_thread' took 147.1403 seconds.
Function 'calculate_frequencies_thread' took 150.7208 seconds.
Function 'merge_frequencies' took 18.6139 seconds.
Function 'merge_frequencies' took 22.4040 seconds.
Function 'merge_frequencies' took 25.5792 seconds.
Function 'merge_frequencies' took 35.0663 seconds.
Function 'merge_frequencies' took 41.0154 seconds.
Function 'merge_frequencies' took 42.1363 seconds.
Function 'merge_frequencies' took 43.2333 seconds.
Function 'merge_frequencies' took 43.4400 seconds.
Top 10 most frequent words:
one: 938677
of: 489253
and: 411784
in: 372201
is: 326973
the: 316376
was: 264976
were: 250430
but: 153544
Function 'print_top_frequencies' took 0.0002 seconds.
Total program execution time: 430.4984 seconds.
Function 'free_words' took 0.4425 seconds.
Function 'free_frequencies' took 0.0013 seconds.
Process returned 0 (0x0)   execution time : 430.638 s
Press ENTER to continue.

```

Conclusion :

The performance analysis of the three approaches—naive, multiprocessing, and multithreading—demonstrates the clear advantages of parallel processing for handling large datasets. The naive approach, being entirely sequential, serves as the baseline and results in the longest execution time, with the lowest speedup and performance metrics.

The multiprocessing approach, which splits the task into multiple independent processes, offers the most significant performance improvements. The optimal performance is achieved with a moderate number of processes, resulting in faster execution times and higher speedups. However, as the number of processes exceeds the system's core capacity, the performance improvement starts to diminish due to the overhead of process management and resource contention.

The multithreading approach also shows improvements in performance, but not as drastically as multiprocessing. Like multiprocessing, the optimal performance occurs with a moderate number of threads, with further increases in threads leading to diminishing returns. This indicates that too many threads can overwhelm the system, causing a drop in efficiency.

